

Задание

Разработать программу для рассадки пассажиров в самолете

Программа получает конфигурацию салона самолета и список пассажиров с предпочтениями по месту. Нужно назначить пассажирам места, вывести результат рассадки и показать статистику занятости

Использовать три модели самолетов условной авиакомпании:

| Код  | Модель      | Бизнес-класс | Схема биз-неса | Эконом-класс | Схема эконома |
|------|-------------|--------------|----------------|--------------|---------------|
| A321 | Airbus A321 | ряды 1-4     | AC-DF          | ряды 5-30    | ABC-DEF       |
| A350 | Airbus A350 | ряды 1-8     | AC-DG-HK       | ряды 9-40    | ABC-DEG-HJK   |
| B777 | Boeing 777  | ряды 1-10    | AC-DG-HK       | ряды 11-45   | ABC-DEFG-HJK  |

Дефис в схеме обозначает проход. Набор мест и их количество зависят от класса обслуживания

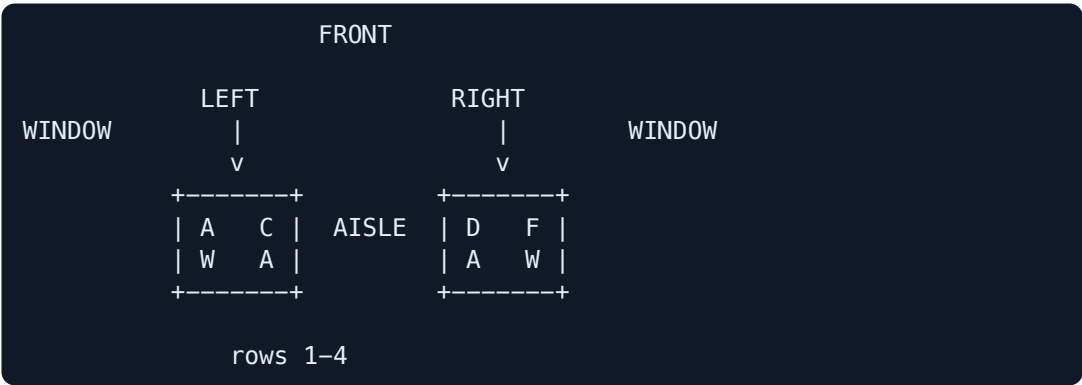
Схемы салонов

Обозначения типов мест:

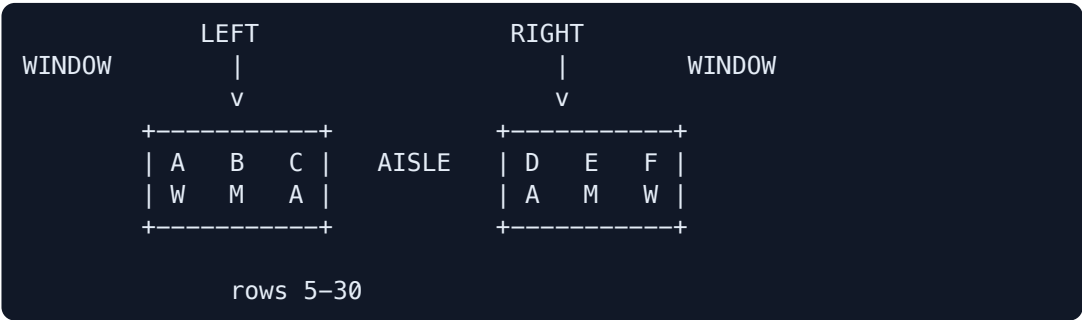
- W — WINDOW
- A — AISLE
- M — MIDDLE

Airbus A321

Бизнес-класс, ряды 1-4, схема 2-2:

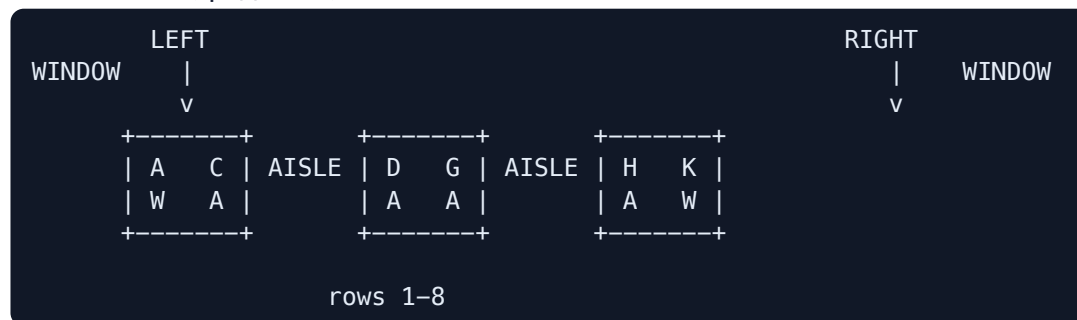


Эконом-класс, ряды 5-30, схема 3-3:

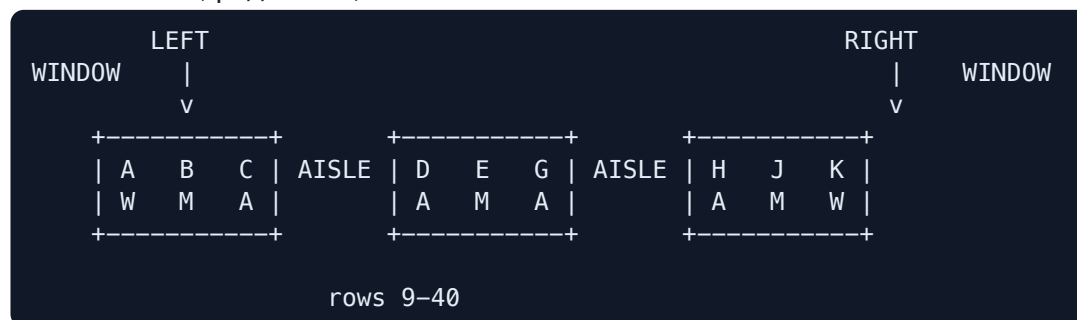


**Airbus A350**

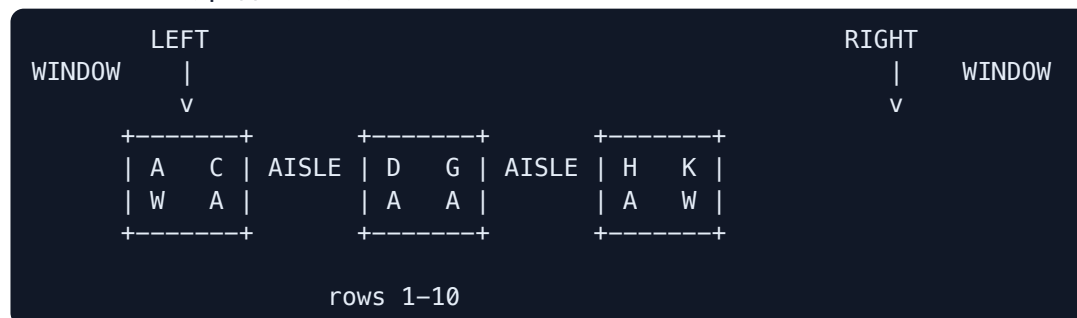
Бизнес-класс, ряды 1-8, схема 2-2-2:



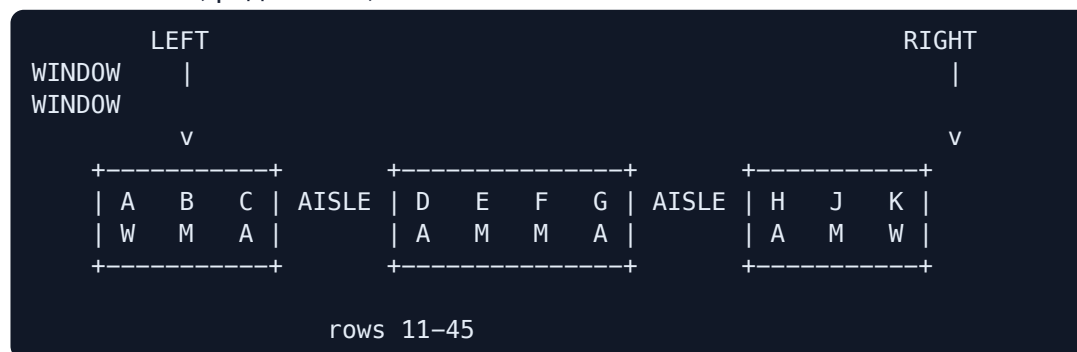
Эконом-класс, ряды 9-40, схема 3-3-3:

**Boeing 777**

Бизнес-класс, ряды 1-10, схема 2-2-2:



Эконом-класс, ряды 11-45, схема 3-4-3:



Места разделяются на блоки проходами. Соседними считаются места одного блока, расположенные рядом друг с другом. Например, в A321 места 12В и 12С соседние, а 12С и 12D разделены проходом

Для места необходимо хранить:

- номер ряда
- букву места
- класс обслуживания
- тип места
- признак занятости

- номер бронирования пассажира, если место занято

Использовать следующие типы мест:

| Тип    | Условие                                        |
|--------|------------------------------------------------|
| WINDOW | крайнее место у внешней стороны салона         |
| AISLE  | место, расположенное непосредственно у прохода |
| MIDDLE | место между двумя другими местами одного блока |

Код самолета и путь к файлу пассажиров передаются через аргументы командной строки

Пример запуска:

```
./seat_allocator A321 passengers.txt
```

Первый аргумент задает код самолета: A321, A350 или B777. Второй аргумент задает путь к файлу пассажиров

Файл `passengers.txt` содержит пассажиров:

```
booking,name,service_class,preference,requested_seat,group_id
BR1001,Ivan Petrov,ECONOMY,WINDOW,,G1
BR1002,Anna Smirnova,ECONOMY,AISLE,12C,
BR1003,Oleg Kim,BUSINESS,WINDOW,,
BR1004,Maria Lee,ECONOMY,TOGETHER,,G1
```

Поля имеют следующий смысл:

| Поле           | Значение                                        |
|----------------|-------------------------------------------------|
| booking        | номер бронирования                              |
| name           | имя пассажира                                   |
| service_class  | BUSINESS или ECONOMY                            |
| preference     | WINDOW, AISLE, MIDDLE, TOGETHER или ANY         |
| requested_seat | конкретное место, например 12C, или пустое поле |
| group_id       | идентификатор группы пассажиров или пустое поле |

Один и тот же файл пассажиров должен обрабатываться для разных моделей самолетов. Если для выбранного самолета мест не хватает или часть мест не подходит, неназначенные пассажиры должны остаться в очереди ожидания

Формат строк можно считать корректным. Если пассажир указал конкретное место, программа должна попытаться назначить его. Место должно существовать в конфигурации выбранного самолета, относиться к указанному классу обслуживания и быть свободным. В остальных случаях пассажир попадает в очередь ожидания

Если конкретное место не указано, программа должна подобрать свободное место по предпочтению. Для пассажиров с одинаковым `group_id` и `preference` TOGETHER нужно попытаться найти соседние места в одном ряду и одном блоке кресел. Достаточно поддерживать группы из 2 или 3 пассажиров

Программа должна использовать последовательные контейнеры по смыслу:

- `std::vector` для хранения мест салона или пассажиров рейса
- `std::array` для фиксированных наборов букв мест в одной из схем салона или фиксированного набора типов мест
- `std::deque` для очереди пассажиров без назначенного места

- `std::list` для истории операций рассадки

В программе должны быть показаны операции добавления, удаления, прохода через `begin` и `end`, а также ситуация, где изменение `std::vector` может сделать ранее полученный `iterator` недействительным

Программа должна вывести:

- выбранную модель самолета
- список назначенных мест в порядке рядов
- список пассажиров без назначенного места
- процент занятости салона
- количество занятых мест бизнес-класса и эконом-класса
- первые 10 свободных мест
- историю операций рассадки

Интерактивный CLI реализовывать не требуется. Достаточно обработать аргументы командной строки и файл пассажиров

## Требования

- Названия переменных, функций, типов и других сущностей программы должны быть осмысленными и записываться на английском языке без транслитерации
- Программа должна читать код самолета и путь к файлу пассажиров из аргументов командной строки
- Программа должна обрабатывать запуск без нужных аргументов командной строки
- Программа должна обрабатывать неизвестный код самолета
- В программе должны использоваться `std::vector`, `std::array`, `std::deque` и `std::list`
- Каждый контейнер должен использоваться для задачи, которая соответствует его модели хранения и операциям
- Программа должна использовать проход по контейнерам через `begin` и `end`
- В программе должен быть отдельный небольшой фрагмент, который демонстрирует `invalidation iterator` для `std::vector`
- Программа должна назначать конкретное место, если оно указано и доступно
- Программа должна автоматически подбирать место по предпочтению пассажира
- Программа должна учитывать разные схемы кресел бизнес-класса и эконом-класса
- Программа должна поддерживать поиск соседних мест для группы из 2 или 3 пассажиров в пределах одного блока кресел
- Программа должна быть грамотно разбита на модули и содержать как минимум один `.hpp` файл и два `.cpp` файла. Допускается использовать больше файлов
- Для заголовочных файлов использовать `#pragma once`
- Программа должна компилироваться и выполняться без ошибок
- Студент должен понимать написанный код, уметь обосновать свое решение и отвечать на вопросы во время защиты лабораторной работы

Минимальный набор операций:

- поиск места по ряду и букве
- чтение пассажиров из входного потока
- назначение пассажира на место

- освобождение места
- поиск свободных мест по классу обслуживания и предпочтению
- поиск соседних мест в одном блоке кресел
- расчет процента занятости салона